

Module 5: Functions Part 2 - Date/Time & Conditional Logic

 **SQL Server Version:** This module has been converted from MySQL to SQL Server (T-SQL) syntax.

This module extends our knowledge of SQL functions by exploring two critical areas: date and time manipulation, and conditional logic. Date/time functions enable us to work with temporal data, calculate durations, and extract specific components from dates. Conditional logic functions allow us to implement decision-making within our queries, handle NULL values, and convert between data types. Together, these tools provide powerful capabilities for data analysis and business logic implementation.

1. Understanding Date and Time Data

Databases store date and time information in specific formats. Understanding these formats and the functions to manipulate them is essential for working with temporal data.

1.1 Common Date/Time Data Types

Data Type	Format	Example	Description
DATE	YYYY-MM-DD	2024-03-15	Stores only the date
TIME	HH:MM:SS	14:30:45	Stores only the time (24-hour format)
DATETIME	YYYY-MM-DD HH:MM:SS	2024-03-15 14:30:45	Stores both date and time
TIMESTAMP	YYYY-MM-DD HH:MM:SS	2024-03-15 14:30:45	Similar to DATETIME but with time zone support

2. Current Date and Time Functions

These functions retrieve the current date and time from the system. They are useful for timestamping records, calculating durations, and comparing with stored dates.

2.1 Getting the Current Date — CAST(GETDATE()) AS DATE

Returns the current date without the time component. In SQL Server, there is no `CURDATE()` or `CURRENT_DATE()`. Use `CAST(GETDATE()) AS DATE` instead:

```
SELECT CAST(GETDATE() AS DATE);
```

Output:

```
+-----+
| CAST(GETDATE() AS DATE) |
+-----+
| 2024-03-15              |
+-----+
1 row in set (0.00 sec)
```

Note (MySQL vs SQL Server): MySQL uses `CURDATE()` or `CURRENT_DATE()`. In SQL Server, use `CAST(GETDATE()) AS DATE` to strip the time portion.

2.2 Getting the Current Time — CAST(GETDATE()) AS TIME

Returns the current time without the date component. SQL Server has no `CURTIME()`; use `CAST(GETDATE()) AS TIME`:

```
SELECT CAST(GETDATE() AS TIME);
```

Output:

```
+-----+
| CAST(GETDATE() AS TIME) |
+-----+
| 14:23:47.0000000        |
+-----+
1 row in set (0.00 sec)
```

Note (MySQL vs SQL Server): MySQL uses `CURTIME()`. In SQL Server, use `CAST(GETDATE()) AS TIME`. You can also use `CONVERT(TIME, GETDATE())` for the same result.

2.3 NOW and CURRENT_TIMESTAMP

Returns both current date and time. In SQL Server, use `GETDATE()` or the standard `CURRENT_TIMESTAMP` (no parentheses):

```
SELECT GETDATE();
```

Output:

```
+-----+
| GETDATE() |
+-----+
| 2024-03-15 14:23:47 |
+-----+
1 row in set (0.00 sec)
```

```
SELECT CURRENT_TIMESTAMP;
```

Output:

```
+-----+
| CURRENT_TIMESTAMP |
+-----+
| 2024-03-15 14:23:47 |
+-----+
1 row in set (0.00 sec)
```

Note (MySQL vs SQL Server): MySQL uses `NOW()` or `CURRENT_TIMESTAMP()` (with parentheses). In SQL Server, `CURRENT_TIMESTAMP` is a keyword (no parentheses), and `GETDATE()` is the most commonly used form.

2.4 Practical Use - Timestamping Records

```
SELECT TOP 5 sname, dob, CAST(GETDATE() AS DATE) AS today
FROM students
```

Output:

```

+-----+-----+-----+
| sname | dob       | today       |
+-----+-----+-----+
| Aarav  | 2010-03-15 | 2024-03-15 |
| Vivaan | 2010-07-22 | 2024-03-15 |
| Aditya | 2009-11-08 | 2024-03-15 |
| Vihaan | 2010-01-30 | 2024-03-15 |
| Arjun  | 2010-09-12 | 2024-03-15 |
+-----+-----+-----+
5 rows in set (0.00 sec)

```

2.5 CURRENT_TIMESTAMP vs GETDATE() — What's the Difference?

Both `CURRENT_TIMESTAMP` and `GETDATE()` return the current date and time, and in most everyday use they produce identical results. However, there are important differences worth knowing:

Feature	GETDATE()	CURRENT_TIMESTAMP
Origin	T-SQL specific (SQL Server only)	ANSI SQL standard (works across databases)
Syntax	Function — requires parentheses	Keyword — no parentheses
Return type	DATETIME	DATETIME
Precision	~3ms (milliseconds)	~3ms (milliseconds)
Timezone awareness	No — returns local server time	No — returns local server time
Portability	SQL Server only	Works in PostgreSQL, MySQL, and others

```

-- Both return the same value:
SELECT GETDATE()          AS from_getdate,
       CURRENT_TIMESTAMP AS from_current_timestamp;

```

Output:

```

+-----+-----+
| from_getdate          | from_current_timestamp |
+-----+-----+

```

```
| 2024-03-15 14:23:47.123 | 2024-03-15 14:23:47.123 |
+-----+-----+
1 row in set (0.00 sec)
```

If you need higher precision (nanoseconds), use `SYSDATETIME()` instead, which returns a `DATETIME2` type:

```
SELECT SYSDATETIME() AS high_precision_now;
```

Output:

```
+-----+
| high_precision_now |
+-----+
| 2024-03-15 14:23:47.1234567 |
+-----+
1 row in set (0.00 sec)
```

Recommendation: Prefer `CURRENT_TIMESTAMP` if you care about writing portable SQL that might run on other databases. Use `GETDATE()` if you are writing SQL Server-only code and want to be explicit about that. For new schemas, consider `DATETIME2` + `SYSDATETIME()` for better precision and range.

2.6 Timezones and UTC — How to Store Dates in Production

Timezone handling is one of the most commonly mishandled areas in database design. SQL Server provides several functions for working with timezones.

2.6.1 Key Timezone Functions in SQL Server

Function	Returns	Timezone Aware?
GETDATE()	DATETIME — server local time	No
GETUTCDATE()	DATETIME — UTC time	No (UTC only)
SYSDATETIMEOFFSET()	DATETIMEOFFSET — includes UTC offset	Yes
AT TIME ZONE		Yes

Function	Returns	Timezone Aware?
	Converts between timezones	

```
-- Get current time in UTC
SELECT GETUTCDATE() AS utc_now;

-- Get current time with timezone offset included
SELECT SYSDATETIMEOFFSET() AS now_with_offset;
```

Output:

```
+-----+
| utc_now          |
+-----+
| 2024-03-15 08:53:47.123 |
+-----+

+-----+
| now_with_offset  |
+-----+
| 2024-03-15 14:23:47.1234567 +05:30 |
+-----+
```

2.6.2 Converting Between Timezones with AT TIME ZONE

```
-- Convert a stored UTC value to Indian Standard Time (IST) for display
SELECT GETUTCDATE() AT TIME ZONE 'UTC'
        AT TIME ZONE 'India Standard Time' AS ist_time;

-- Convert to US Eastern Time
SELECT GETUTCDATE() AT TIME ZONE 'UTC'
        AT TIME ZONE 'Eastern Standard Time' AS us_eastern;
```

Output:

```
+-----+
| ist_time          |
+-----+
| 2024-03-15 14:23:47.000 +05:30 |
+-----+

+-----+
| us_eastern        |
+-----+
```

```
+-----+
| 2024-03-15 03:53:47.000 -05:00 |
+-----+
```

2.6.3 Should You Store Dates in UTC? — Production Recommendation

Yes — storing all dates in UTC on the server and letting the front end handle timezone formatting is the correct, industry-standard approach for production systems. Here is why:

- **No ambiguity at DST boundaries:** Daylight Saving Time transitions can cause local times to repeat (e.g., 1:30 AM happens twice). UTC never has this problem.
- **Consistent sorting and comparison:** UTC values sort correctly without any timezone conversion logic in your queries.
- **Multi-region support:** If your users are in different countries, storing UTC means every user can see the same event displayed in their own local time, without any data duplication.
- **Simpler server-side logic:** All calculations (age, duration, overdue, etc.) are straightforward when everything is in the same timezone.
- **Portability:** Migrations between servers in different regions don't corrupt your data.

```
-- GOOD production pattern: store in UTC
INSERT INTO orders (order_date, customer_id)
VALUES (GETUTCDATE(), 42);

-- GOOD production pattern: retrieve UTC and convert in the SELECT for display
SELECT order_id,
       order_date AS stored_utc,
       order_date AT TIME ZONE 'UTC'
                AT TIME ZONE 'India Standard Time' AS display_for_ist_user
FROM orders;
```

Output:

```
+-----+-----+-----+
| order_id | stored_utc          | display_for_ist_user          |
+-----+-----+-----+
|      1045 | 2024-03-15 08:53:47.000 | 2024-03-15 14:23:47.000 +05:30 |
|      1046 | 2024-03-15 10:15:22.000 | 2024-03-15 15:45:22.000 +05:30 |
+-----+-----+-----+
```

The front end then reads the `+05:30` offset and renders it however it likes — no timezone logic needs to live in your SQL at all for routine reads.

2.6.4 What Data Type Should You Use?

Data Type	Stores Timezone?	Precision	Recommendation
DATETIME	No	~3ms	Legacy; avoid in new schemas
DATETIME2	No	100ns	Good for UTC storage (pair with GETUTCDATE() / SYSUTCDATETIME())
DATETIMEOFFSET	Yes (as offset)	100ns	Best when you need to preserve the user's original local time alongside UTC

Practical rule: For most production systems, store as `DATETIME2` in UTC using `SYSUTCDATETIME()` . Reserve `DATETIMEOFFSET` for cases where knowing the original local timezone of an event matters (e.g., a calendar app where "the meeting was scheduled for 9 AM London time" needs to be preserved even if London changes its clock).

3. Extracting Date Components

These functions extract specific parts from a date or datetime value, such as the year, month, day, hour, minute, or second.

3.1 YEAR Function

```
SELECT YEAR('2024-03-15');
```

Output:

```
+-----+
| YEAR('2024-03-15') |
+-----+
|          2024        |
+-----+
1 row in set (0.00 sec)
```


3.2 MONTH Function

```
SELECT MONTH('2024-03-15');
```

Output:

```
+-----+
| MONTH('2024-03-15') |
+-----+
|          3          |
+-----+
1 row in set (0.00 sec)
```

3.3 DAY Function

```
SELECT DAY('2024-03-15');
```

Output:

```
+-----+
| DAY('2024-03-15') |
+-----+
|          15        |
+-----+
1 row in set (0.00 sec)
```

3.4 DATEPART for Hour, Minute, and Second

SQL Server does not have standalone `HOUR()`, `MINUTE()`, or `SECOND()` functions like MySQL. Instead, use `DATEPART()` with the appropriate interval keyword:

```
SELECT DATEPART(hour, '2024-03-15 14:23:47');
```

Output:

```
+-----+
| DATEPART(hour, '2024-03-15 14:23:47') |
+-----+
```

	14
1 row in set (0.00 sec)	

```
SELECT DATEPART(minute, '2024-03-15 14:23:47'),
       DATEPART(second, '2024-03-15 14:23:47');
```

Output:

DATEPART(minute, '2024-03-15 14:23:47')	DATEPART(second, '2024-03-15 14:23:47')
23	47

1 row in set (0.00 sec)

Note (MySQL vs SQL Server): MySQL has `HOUR('14:23:47')`, `MINUTE('14:23:47')`, `SECOND('14:23:47')`. In SQL Server, these do not exist — use `DATEPART(hour, ...)`, `DATEPART(minute, ...)`, and `DATEPART(second, ...)`. Note that SQL Server also requires a full datetime value, not a bare time string.

3.5 Practical Example - Analyzing Birth Months

```
SELECT MONTH(dob) AS birth_month, COUNT(*) AS student_count
FROM students
GROUP BY MONTH(dob)
ORDER BY birth_month;
```

Output:

birth_month	student_count
1	38
2	35
3	42

4	39
5	36
6	41
7	37
8	40
9	38
10	35
11	34
12	35

```

+-----+
12 rows in set (0.01 sec)

```

This query groups students by their birth month and counts how many students were born in each month.

3.6 Finding Students Born in Specific Year

```

SELECT TOP 5 sname, dob, YEAR(dob) AS birth_year
FROM students
WHERE YEAR(dob) = 2010

```

Output:

sname	dob	birth_year
Aarav	2010-03-15	2010
Vivaan	2010-07-22	2010
Vihaan	2010-01-30	2010
Arjun	2010-09-12	2010
Sai	2010-05-18	2010

```

+-----+
5 rows in set (0.01 sec)

```

4. DATENAME — Getting Day and Month Names

SQL Server uses `DATENAME()` to return the name of a date part as a string. MySQL uses separate functions `DAYNAME()` and `MONTHNAME()`, which do not exist in SQL Server.

4.1 Day Name — DATENAME(weekday, ...)

```
SELECT DATENAME(weekday, '2024-03-15');
```

Output:

```
+-----+
| DATENAME(weekday, '2024-03-15') |
+-----+
| Friday                            |
+-----+
1 row in set (0.00 sec)
```

Note (MySQL vs SQL Server): MySQL uses `DAYNAME('2024-03-15')`. In SQL Server, use `DATENAME(weekday, '2024-03-15')`.

4.2 Month Name — DATENAME(month, ...)

```
SELECT DATENAME(month, '2024-03-15');
```

Output:

```
+-----+
| DATENAME(month, '2024-03-15') |
+-----+
| March                          |
+-----+
1 row in set (0.00 sec)
```

Note (MySQL vs SQL Server): MySQL uses `MONTHNAME('2024-03-15')`. In SQL Server, use `DATENAME(month, '2024-03-15')`.

4.3 Practical Example - Birth Day Analysis

```
SELECT TOP 5 sname, dob, DATENAME(weekday, dob) AS birth_day
FROM students
```

Output:

```
+-----+-----+-----+
| sname | dob       | birth_day |
+-----+-----+-----+
| Aarav  | 2010-03-15 | Monday    |
| Vivaan | 2010-07-22 | Thursday  |
| Aditya | 2009-11-08 | Sunday    |
| Vihaan | 2010-01-30 | Saturday  |
| Arjun  | 2010-09-12 | Sunday    |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

4.4 Counting Births by Day of Week

```
SELECT DATENAME(weekday, dob) AS day_of_week, COUNT(*) AS count
FROM students
GROUP BY DATENAME(weekday, dob)
ORDER BY count DESC;
```

Output:

```
+-----+-----+
| day_of_week | count |
+-----+-----+
| Saturday    | 68    |
| Sunday      | 67    |
| Friday      | 66    |
| Tuesday     | 65    |
| Wednesday   | 64    |
| Thursday    | 63    |
| Monday      | 57    |
+-----+-----+
7 rows in set (0.01 sec)
```

5. DATEADD Function

SQL Server uses `DATEADD()` to add or subtract time intervals from a date. MySQL uses `DATE_ADD()` and `DATE_SUB()` with `INTERVAL` syntax, which are **not available** in SQL Server.

5.4 Adding Years

```
SELECT DATEADD(year, 2, '2024-03-15');
```

Output:

```
+-----+
| DATEADD(year, 2, '2024-03-15') |
+-----+
| 2026-03-15                      |
+-----+
1 row in set (0.00 sec)
```

5.5 Subtracting Dates — DATEADD with a Negative Number

To subtract, use a negative value in `DATEADD()`. There is no separate `DATE_SUB()` in SQL Server:

```
SELECT DATEADD(day, -7, '2024-03-15');
```

Output:

```
+-----+
| DATEADD(day, -7, '2024-03-15') |
+-----+
| 2024-03-08                      |
+-----+
1 row in set (0.00 sec)
```

Note (MySQL vs SQL Server): MySQL uses `DATE_SUB('2024-03-15', INTERVAL 7 DAY)`. In SQL Server, use `DATEADD(day, -7, '2024-03-15')` — a negative number subtracts.

5.6 Practical Example - Calculating Future Dates

```
SELECT TOP 5 sname, dob,
       DATEADD(year, 18, dob) AS becomes_adult
FROM students
```

Output:

```
+-----+-----+-----+
| sname | dob       | becomes_adult |
+-----+-----+-----+
| Aarav  | 2010-03-15 | 2028-03-15    |
| Vivaan | 2010-07-22 | 2028-07-22    |
| Aditya | 2009-11-08 | 2027-11-08    |
| Vihaan | 2010-01-30 | 2028-01-30    |
| Arjun  | 2010-09-12 | 2028-09-12    |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

5.7 Finding Records from Last 30 Days

```
SELECT order_id, order_date
FROM orders
WHERE order_date >= DATEADD(day, -30, CAST(GETDATE() AS DATE));
```

Output:

```
+-----+-----+
| order_id | order_date |
+-----+-----+
| 1045     | 2024-03-01 |
| 1046     | 2024-03-05 |
| 1047     | 2024-03-12 |
| 1048     | 2024-03-14 |
+-----+-----+
4 rows in set (0.01 sec)
```

This retrieves all orders placed in the last 30 days.

6. DATEDIFF Function

The DATEDIFF function calculates the number of days between two dates. This is useful for calculating age, duration, elapsed time, or days remaining.

6.1 DATEDIFF Syntax

```
DATEDIFF(day, start_date, end_date)
```

Returns the number of days between the two dates. The result is positive if end_date is after start_date, and negative otherwise.

6.2 Basic DATEDIFF Example

```
SELECT DATEDIFF(day, '2024-03-01', '2024-03-15');
```

Output:

```
+-----+
| DATEDIFF('2024-03-15', '2024-03-01') |
+-----+
|                14                |
+-----+
1 row in set (0.00 sec)
```

There are 14 days between March 1st and March 15th, 2024.

6.3 Calculating Age in Days

```
SELECT TOP 5 sname, dob,
      DATEDIFF(day, dob, CAST(GETDATE() AS DATE)) AS age_in_days
FROM students
```

Output:

```
+-----+-----+-----+
| sname  | dob      | age_in_days |
+-----+-----+-----+
| Aarav  | 2010-03-15 | 5114        |
| Vivaan | 2010-07-22 | 4985        |
| Aditya | 2009-11-08 | 5241        |
| Vihaan | 2010-01-30 | 5158        |
```

```
| Arjun | 2010-09-12 | 4933 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

6.4 Calculating Age in Years

```
SELECT TOP 5 sname, dob,
             FLOOR(DATEDIFF(day, dob, CAST(GETDATE() AS DATE)) / 365.25) AS age_in_years
FROM students
```

Output:

```
+-----+-----+-----+
| sname | dob       | age_in_years |
+-----+-----+-----+
| Aarav | 2010-03-15 | 14           |
| Vivaan | 2010-07-22 | 13           |
| Aditya | 2009-11-08 | 14           |
| Vihaan | 2010-01-30 | 14           |
| Arjun | 2010-09-12 | 13           |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

We divide by 365.25 to account for leap years, and use FLOOR to get complete years.

6.5 Finding Overdue Items

```
SELECT book_id, due_date,
       DATEDIFF(day, due_date, CAST(GETDATE() AS DATE)) AS days_overdue
FROM library_books
WHERE due_date < CAST(GETDATE() AS DATE);
```

Output:

```
+-----+-----+-----+
| book_id | due_date | days_overdue |
+-----+-----+-----+
| B101 | 2024-03-01 | 14           |
| B205 | 2024-03-10 | 5            |
+-----+-----+-----+
```

| B312 | 2024-02-28 | 16 |

+-----+-----+-----+

3 rows in set (0.00 sec)

6.6 Date Units Reference — DATEPART, DATEDIFF, and DATEADD

The same set of unit keywords is used across `DATEPART()`, `DATEDIFF()`, and `DATEADD()`. SQL Server accepts both the full word and a short abbreviation — both work identically.

Unit (full)	Abbreviation	Range / Notes	Example — DATEPART	Example — DATEADD / DATEDIFF
year	yy, yyyy	1753 – 9999	DATEPART(year, '2024-03-15') → 2024	DATEADD(year, 1, '2024-03-15') → 2025-03-15
quarter	qq, q	1 – 4	DATEPART(qua rter, '2024-03-15') → 1	DATEDIFF(qua rter, '2024-01-01', '2024-09-01') → 2
month	mm, m	1 – 12	DATEPART(mo nth, '2024-03-15') → 3	DATEADD(mon th, 3, '2024-03-15') → 2024-06-15
dayofyear	dy, y	1 – 366	DATEPART(day ofyear, '2024-03-15') → 75	DATEDIFF(day ofyear, '2024-01-01', '2024-03-15') → 74
day	dd, d	1 – 31	DATEPART(day, '2024-03-15') → 15	DATEADD(day, 10, '2024-03-15') → 2024-03-25
week	wk, ww	1 – 53		

Unit (full)	Abbreviation	Range / Notes	Example — DATEPART	Example — DATEADD / DATEDIFF
			DATEPART(week, '2024-03-15') → 11	DATEADD(week, 2, '2024-03-15') → 2024-03-29
weekday	dw	1 (Sunday) – 7 (Saturday) by default	DATEPART(weekday, '2024-03-15') → 6 (Friday)	Not typically used with DATEADD/ DATEDIFF
hour	hh	0 – 23	DATEPART(hour, '2024-03-15 14:23:47') → 14	DATEADD(hour, 5, '2024-03-15 14:00:00') → 2024-03-15 19:00:00
minute	mi, n	0 – 59	DATEPART(minute, '2024-03-15 14:23:47') → 23	DATEADD(minute, 30, '2024-03-15 14:00:00') → 2024-03-15 14:30:00
second	ss, s	0 – 59	DATEPART(second, '2024-03-15 14:23:47') → 47	DATEADD(second, 45, '2024-03-15 14:23:00') → 2024-03-15 14:23:45
millisecond	ms	0 – 999	DATEPART(millisecond, '2024-03-15 14:23:47.123') → 123	DATEADD(millisecond, 500, '2024-03-15 14:23:47') → 2024-03-15 14:23:47.500
microsecond	mcs	0 – 999999 (DATETIME2 only)		DATEADD(microsecond, 500, SYSDATETIME())

Unit (full)	Abbreviation	Range / Notes	Example — DATEPART	Example — DATEADD / DATEDIFF
			DATEPART(microsecond, SYSDATETIME()) → e.g. 123456	
nanosecond	ns	0 – 999999999 (DATETIME2 only)	DATEPART(nanosecond, SYSDATETIME()) → e.g. 1234567	DATEADD(nanosecond, 100, SYSDATETIME())
tzoffset	tz	-840 to +840 minutes (DATETIMEOFFSET only)	DATEPART(tzoffset, SYSDATETIMEOFFSET()) → e.g. 330 (= +05:30)	Not applicable with DATEADD/ DATEDIFF
iso_week	isowk, isoww	1 – 53 (ISO 8601 week numbering)	DATEPART(iso_week, '2024-01-01') → 1	Not applicable with DATEADD/ DATEDIFF

6.6.1 Practical Examples Using Different Units

```
-- How many weeks old is each student?
SELECT TOP 5 sname, dob,
      DATEDIFF(week, dob, CAST(GETDATE() AS DATE)) AS age_in_weeks
FROM students
```

Output:

sname	dob	age_in_weeks
Aarav	2010-03-15	731
Vivaan	2010-07-22	712
Aditya	2009-11-08	749
Vihaan	2010-01-30	737

```
| Arjun | 2010-09-12 | 705 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
-- Extract the quarter from a date (useful for financial reporting)
SELECT sname, dob,
       DATEPART(quarter, dob) AS birth_quarter
FROM students
WHERE DATEPART(year, dob) = 2010;
```

Output:

```
+-----+-----+-----+
| sname | dob       | birth_quarter |
+-----+-----+-----+
| Aarav | 2010-03-15 | 1             |
| Vivaan | 2010-07-22 | 3             |
| Vihaan | 2010-01-30 | 1             |
| Arjun | 2010-09-12 | 3             |
| Sai   | 2010-05-18 | 2             |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
-- Add 90 minutes to a meeting time (useful for scheduling)
SELECT DATEADD(minute, 90, '2024-03-15 09:00:00') AS meeting_end;
```

Output:

```
+-----+
| meeting_end |
+-----+
| 2024-03-15 10:30:00 |
+-----+
1 row in set (0.00 sec)
```

6.6.2 Important Caveats

- **DATEDIFF counts boundary crossings, not elapsed time:** `DATEDIFF(year, '2024-12-31', '2025-01-01')` returns `1` — it crossed a year boundary — even though only 1 day passed. Always verify which unit truly fits your logic.
- **microsecond and nanosecond require DATETIME2 or DATETIMEOFFSET:** They will error on plain DATETIME columns.
- **weekday numbering depends on DATEFIRST setting:** By default Sunday = 1, Saturday = 7, but this can be changed with `SET DATEFIRST`. Use `DATENAME(weekday, date)` to get the name if consistency matters more than the number.
- **iso_week vs week:** ISO week numbering (Monday-first, week 1 = first week with a Thursday) differs from the default SQL Server `week`. For international applications, prefer `iso_week`.

7. FORMAT Function

SQL Server uses the `FORMAT()` function to convert a date into a formatted string. MySQL uses `DATE_FORMAT()` with `%`-style specifiers; SQL Server uses `FORMAT()` with .NET-style format strings.

7.1 FORMAT Syntax

```
FORMAT(value, format_string [, culture])
```

Note (MySQL vs SQL Server): MySQL uses `DATE_FORMAT(date, '%M %d, %Y')`. In SQL Server, the equivalent is `FORMAT(date, 'MMM dd, yyyy')`. The format specifiers are completely different.

7.2 Common Format Specifiers (SQL Server / .NET style)

SQL Server Specifier	MySQL Equivalent	Description	Example Output
yyyy	%Y	Year (4 digits)	2024
yy	%y	Year (2 digits)	24
MMMM	%M	Full month name	March
MM	%m	Month (zero-padded numeric)	03
dd	%d	Day (zero-padded numeric)	15

SQL Server Specifier	MySQL Equivalent	Description	Example Output
dddd	%W	Full weekday name	Friday
HH	%H	Hour (00–23, zero-padded)	14
mm	%i	Minutes (zero-padded)	23
ss	%s	Seconds (zero-padded)	47

Note on ordinal suffixes (e.g., "15th"): MySQL's `%D` adds ordinal suffixes automatically. SQL Server's `FORMAT()` has no equivalent — you need a custom `CASE` expression if ordinal suffixes are required.

7.3 Formatting Examples

```
SELECT FORMAT('2024-03-15', 'MMMM dd, yyyy');
```

Output:

```
+-----+
| FORMAT('2024-03-15', 'MMMM dd, yyyy') |
+-----+
| March 15, 2024                          |
+-----+
1 row in set (0.00 sec)
```

```
SELECT FORMAT('2024-03-15', 'dd/MM/yyyy');
```

Output:

```
+-----+
| FORMAT('2024-03-15', 'dd/MM/yyyy') |
+-----+
```



```
| 15/03/2024 |
+-----+
1 row in set (0.00 sec)
```

```
SELECT FORMAT(GETDATE(), 'dddd, MMMM dd, yyyy HH:mm');
```

Output:

```
+-----+
| FORMAT(GETDATE(), 'dddd, MMMM dd, yyyy HH:mm') |
+-----+
| Friday, March 15, 2024 14:23 |
+-----+
1 row in set (0.00 sec)
```

7.4 Practical Example - Report Formatting

```
SELECT TOP 5 sname,
             FORMAT(dob, 'MMMM dd, yyyy') AS formatted_dob,
             FORMAT(dob, 'dddd') AS birth_day
FROM students
```

Output:

```
+-----+-----+-----+
| sname | formatted_dob | birth_day |
+-----+-----+-----+
| Aarav | March 15, 2010 | Monday   |
| Vivaan | July 22, 2010 | Thursday |
| Aditya | November 08, 2009 | Sunday  |
| Vihaan | January 30, 2010 | Saturday |
| Arjun | September 12, 2010 | Sunday  |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

8. CASE WHEN Statement

The CASE statement implements conditional logic in SQL. It allows you to perform different actions based on different conditions, similar to if-else statements in programming languages.

8.1 Simple CASE Syntax

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE default_result
END
```

8.2 Basic CASE Example

```
SELECT TOP 8 sname, maths,
  CASE
    WHEN maths >= 90 THEN 'Excellent'
    WHEN maths >= 75 THEN 'Good'
    WHEN maths >= 60 THEN 'Average'
    WHEN maths >= 35 THEN 'Pass'
    ELSE 'Fail'
  END AS grade
FROM students
```

Output:

sname	maths	grade
Aarav	85	Good
Vivaan	72	Average
Aditya	88	Good
Vihaan	65	Average
Arjun	91	Excellent
Sai	58	Pass
Arnav	78	Good
Krishna	82	Good

8 rows in set (0.01 sec)

The CASE statement evaluates conditions from top to bottom and returns the result of the first true condition.

8.3 CASE with Multiple Conditions

```
SELECT TOP 5 sname, english, maths, science,
CASE
    WHEN english >= 35 AND maths >= 35 AND science >= 35 THEN 'Pass'
    ELSE 'Fail'
END AS overall_result
FROM students
```

Output:

sname	english	maths	science	overall_result
Aarav	78	85	82	Pass
Vivaan	74	72	83	Pass
Aditya	80	88	78	Pass
Vihaan	72	65	83	Pass
Arjun	75	91	73	Pass

5 rows in set (0.00 sec)

8.4 CASE for Data Categorization

```
SELECT TOP 5 sname,
DATEDIFF(day, dob, CAST(GETDATE() AS DATE)) / 365.25 AS age,
CASE
    WHEN DATEDIFF(day, dob, CAST(GETDATE() AS DATE)) / 365.25 < 13 THEN 'Child'
    WHEN DATEDIFF(day, dob, CAST(GETDATE() AS DATE)) / 365.25 < 18 THEN
'Teenager'
    ELSE 'Adult'
END AS age_category
FROM students
```

Output:

sname	age	age_category
Aarav	14.003425	Teenager
Vivaan	13.650685	Teenager
Aditya	14.351233	Teenager
Vihaan	14.126712	Teenager

Arjun	13.508219	Teenager
-------	-----------	----------

5 rows in set (0.01 sec)

8.5 CASE in WHERE Clause

You can also use CASE in WHERE clauses for dynamic filtering:

```
SELECT TOP 5 sname, standard, division
FROM students
WHERE
CASE
    WHEN standard = 10 THEN division IN ('A', 'B')
    ELSE division = 'A'
END
```

Output:

sname	standard	division
Aarav	10	A
Vivaan	9	A
Aditya	10	B
Arjun	10	A
Krishna	8	A

5 rows in set (0.01 sec)

8.6 Nested CASE Statements

```
SELECT TOP 5 sname, standard,
    (english + maths + science) / 3 AS average,
    CASE
        WHEN standard >= 8 THEN
            CASE
                WHEN (english + maths + science) / 3 >= 80 THEN 'Senior Excellent'
                WHEN (english + maths + science) / 3 >= 60 THEN 'Senior Good'
                ELSE 'Senior Average'
            END
        ELSE
            CASE
                WHEN (english + maths + science) / 3 >= 80 THEN 'Junior Excellent'
                WHEN (english + maths + science) / 3 >= 60 THEN 'Junior Good'
```

```

ELSE 'Junior Average'
END
END AS category
FROM students

```

Output:

```

+-----+-----+-----+-----+
| sname | standard | average | category |
+-----+-----+-----+-----+
| Aarav | 10 | 81.67 | Senior Excellent |
| Vivaan | 9 | 76.33 | Senior Good |
| Aditya | 10 | 82.00 | Senior Excellent |
| Vihaan | 7 | 73.33 | Junior Good |
| Arjun | 10 | 79.67 | Senior Good |
+-----+-----+-----+-----+
5 rows in set (0.01 sec)

```

9. NULL Handling Functions

NULL represents the absence of a value. It's different from zero or an empty string. Handling NULL values properly is crucial for accurate data analysis.

9.1 Understanding NULL

```

SELECT TOP 3 * FROM students
WHERE marks IS NULL

```

Output:

```

+-----+-----+-----+-----+-----+-----+-----+
| roll | sname | english | maths | science | standard | division |
+-----+-----+-----+-----+-----+-----+-----+
| 45 | Nidhi | NULL | NULL | NULL | 8 | C |
| 67 | Rohan | NULL | 75 | 82 | 9 | A |
| 89 | Kavya | 78 | NULL | 85 | 10 | B |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)

```

Note: We use IS NULL, not = NULL, to check for NULL values.

10. ISNULL and COALESCE Functions

These functions replace NULL values with a specified alternative value.

10.1 ISNULL Function

ISNULL takes two arguments and returns the first one if it's not NULL, otherwise returns the second:

```
SELECT ISNULL(NULL, 'Default Value');
```

Output:

```
+-----+
| ISNULL(NULL, 'Default Value') |
+-----+
| Default Value                  |
+-----+
1 row in set (0.00 sec)
```

```
SELECT ISNULL('Actual Value', 'Default Value');
```

Output:

[illegible]

10.2 Practical ISNULL Example

```
SELECT TOP 5 sname,  
            ISNULL(maths, 0) AS maths_score,  
            ISNULL(english, 0) AS english_score  
FROM students  
WHERE maths IS NULL OR english IS NULL
```

Output:

sname	maths_score	english_score
Nidhi	0	0
Rohan	75	0
Kavya	0	78
Aryan	0	82
Tanvi	68	0

5 rows in set (0.01 sec)

This replaces NULL marks with 0, which might be useful for calculations.

10.3 COALESCE Function

COALESCE can take multiple arguments and returns the first non-NULL value:

```
SELECT COALESCE(NULL, NULL, 'First Non-NULL', 'Second');
```

Output:

COALESCE(NULL, NULL, 'First Non-NULL', 'Second')
First Non-NULL

1 row in set (0.00 sec)

10.4 COALESCE with Multiple Columns

```
SELECT TOP 5 sname,  
             COALESCE(maths, english, science, 0) AS first_available_marks  
FROM students
```

Output:

sname	first_available_marks
Aarav	85
Vivaan	72
Aditya	88
Vihaan	65
Arjun	91

5 rows in set (0.00 sec)

COALESCE returns the first non-NULL value from the list of columns, or 0 if all are NULL.

10.5 Using COALESCE for Default Addresses

```
SELECT TOP 3 sname,  
             COALESCE(address, 'Address Not Provided') AS student_address  
FROM students  
WHERE address IS NULL OR address = ''
```

Output:

sname	student_address
Nidhi	Address Not Provided
Rohan	Address Not Provided
Kavya	Address Not Provided

3 rows in set (0.00 sec)

11. NULLIF Function

The NULLIF function returns NULL if two expressions are equal, otherwise it returns the first expression. This is useful for preventing division by zero errors or converting specific values to NULL.

11.1 NULLIF Syntax

```
NULLIF(expression1, expression2)
```

11.2 Basic NULLIF Example

```
SELECT NULLIF(10, 10);
```

Output:

```
+-----+
| NULLIF(10, 10) |
+-----+
| NULL           |
+-----+
1 row in set (0.00 sec)
```

```
SELECT NULLIF(10, 5);
```

Output:

```
+-----+
| NULLIF(10, 5) |
+-----+
|          10   |
+-----+
1 row in set (0.00 sec)
```

11.3 Preventing Division by Zero

```
SELECT 100 / NULLIF(0, 0) AS safe_division;
```

Output:

```
+-----+
| safe_division |
+-----+
| NULL          |
+-----+
1 row in set (0.00 sec)
```

Instead of causing a division by zero error, this returns NULL.

11.4 Practical Example

```
SELECT TOP 5 sname,
             maths,
             science,
             ROUND(100 / NULLIF(science, 0) * maths, 2) AS maths_percentage_of_science
FROM students
```

Output:

```
+-----+-----+-----+-----+
| sname  | maths | science | maths_percentage_of_science |
+-----+-----+-----+-----+
| Aarav  | 85    | 82      | 103.66                       |
| Vivaan | 72    | 83      | 86.75                        |
| Aditya | 88    | 78      | 112.82                       |
| Vihaan | 65    | 83      | 78.31                        |
| Arjun  | 91    | 73      | 124.66                       |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

12. Type Conversion Functions

Type conversion functions convert data from one type to another. This is essential when working with data from different sources or when performing operations that require specific data types.

13. CAST Function

The CAST function converts a value from one data type to another using standard SQL syntax.

13.1 CAST Syntax

```
CAST(expression AS data_type)
```

13.2 Converting to Integer

```
SELECT CAST('123' AS INT);
```

Output:

```
+-----+
| CAST('123' AS INT) |
+-----+
|          123       |
+-----+
1 row in set (0.00 sec)
```

The string '123' is converted to the integer 123. **Note (MySQL vs SQL Server):** MySQL supports `CAST(... AS UNSIGNED)`. SQL Server does not have an UNSIGNED type — use `INT`, `BIGINT`, or `TINYINT` depending on the size needed.

13.3 Converting to Decimal

```
SELECT CAST('123.456' AS DECIMAL(10, 2));
```

Output:

```

+-----+
| CAST('123.456' AS DECIMAL(10, 2)) |
+-----+
|                123.46                |
+-----+
1 row in set (0.00 sec)

```

13.4 Converting to Date

```
SELECT CAST('2024-03-15' AS DATE);
```

Output:

```

+-----+
| CAST('2024-03-15' AS DATE) |
+-----+
|          2024-03-15          |
+-----+
1 row in set (0.00 sec)

```

13.5 Practical Example - Sorting String Numbers

```

SELECT TOP 5 roll, sname
FROM students
WHERE CAST(roll AS VARCHAR(10)) LIKE '1%'
ORDER BY CAST(roll AS INT)

```

Output:

```

+-----+-----+
| roll | sname |
+-----+-----+
| 1    | Aarav |
| 10   | Ananya|
| 11   | Ishaan|
| 12   | Diya  |

```

```
| 13 | Kabir |
+-----+-----+
5 rows in set (0.01 sec)
```

14. CONVERT Function

CONVERT is an alternative to CAST for type conversion. **Important:** The argument order is different between MySQL and SQL Server. In MySQL, the expression comes first; in SQL Server, the target data type comes first.

14.1 CONVERT Syntax (SQL Server)

```
CONVERT(data_type [(length)], expression [, style])
```

Note (MySQL vs SQL Server): MySQL syntax is `CONVERT(expression, data_type)`. SQL Server reverses this to `CONVERT(data_type, expression)`. This is a common source of errors when porting queries.

14.2 Basic CONVERT Examples

```
SELECT CONVERT(INT, '123');
```

Output:

```
+-----+
| CONVERT(INT, '123') |
+-----+
| 123 |
+-----+
1 row in set (0.00 sec)
```

```
SELECT CONVERT(DATE, '2024-03-15');
```

Output:

```

+-----+
| CONVERT(DATE, '2024-03-15') |
+-----+
|          2024-03-15          |
+-----+
1 row in set (0.00 sec)

```

14.3 Converting for Calculations

```

SELECT TOP 5 sname,
             CAST((english + maths + science) / 3.0 AS DECIMAL(5,2)) AS average,
             CONVERT(VARCHAR, CAST((english + maths + science) / 3.0 AS DECIMAL(5,2))) + '%'
AS average_percentage
FROM students

```

Output:

```

+-----+-----+-----+
| sname  | average | average_percentage |
+-----+-----+-----+
| Aarav  | 81.67   | 81.67%             |
| Vivaan | 76.33   | 76.33%             |
| Aditya | 82.00   | 82.00%             |
| Vihaan | 73.33   | 73.33%             |
| Arjun  | 79.67   | 79.67%             |
+-----+-----+-----+
5 rows in set (0.00 sec)

```

15. Combining Functions for Complex Queries

The real power of SQL functions emerges when we combine them to solve complex problems.

15.1 Student Report Card

```

SELECT TOP 5 sname,
             FORMAT(dob, 'MMMM dd, yyyy') AS birth_date,
             FLOOR(DATEDIFF(day, dob, CAST(GETDATE() AS DATE)) / 365.25) AS age,
             ISNULL(english, 0) AS eng,
             ISNULL(maths, 0) AS math,
             ISNULL(science, 0) AS sci,
             COALESCE(english, 0) + COALESCE(maths, 0) + COALESCE(science, 0) AS total,

```

```

CASE
  WHEN COALESCE(english, 0) >= 35
    AND COALESCE(maths, 0) >= 35
    AND COALESCE(science, 0) >= 35
  THEN 'PASS'
  ELSE 'FAIL'
END AS result
FROM students

```

Output:

```

+-----+-----+-----+-----+-----+-----+-----+
| sname   | birth_date   | age | eng | math | sci | total | result |
+-----+-----+-----+-----+-----+-----+-----+
| Aarav   | March 15, 2010 | 14 | 78 | 85 | 82 | 245 | PASS |
| Vivaan  | July 22, 2010  | 13 | 74 | 72 | 83 | 229 | PASS |
| Aditya  | November 08, 2009 | 14 | 80 | 88 | 78 | 246 | PASS |
| Vihaan  | January 30, 2010 | 14 | 72 | 65 | 83 | 220 | PASS |
| Arjun   | September 12, 2010 | 13 | 75 | 91 | 73 | 239 | PASS |
+-----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.01 sec)

```

15.2 Employee Salary Analysis

```

SELECT ename,
  CAST(esalary AS INT) AS current_salary,
  CAST(esalary * 1.10 AS INT) AS after_10_percent_hike,
  CASE
    WHEN esalary < 20000 THEN 'Entry Level'
    WHEN esalary < 40000 THEN 'Mid Level'
    ELSE 'Senior Level'
  END AS salary_band,
  FORMAT DATEADD(year, 1, CAST(GETDATE() AS DATE)), 'MMMM yyyy') AS next_review
FROM employees;

```

Output:

```

+-----+-----+-----+-----+-----+
| ename   | current_salary | after_10_percent_hike | salary_band | next_review |
+-----+-----+-----+-----+-----+
| Arun    | 45000          | 49500                  | Senior Level | March 2025 |
| Vidhi   | 15000          | 16500                  | Entry Level  | March      |

```

```

2025 |
| Amit | 25000 | 27500 | Mid Level | March
2025 |
| Arvind | 55000 | 60500 | Senior Level | March
2025 |
+-----+-----+-----+-----+
+-----+
4 rows in set (0.00 sec)

```

15.3 Date-Based Conditional Formatting

```

SELECT TOP 8 sname, dob,
    DATEDIFF(day, dob, CAST(GETDATE() AS DATE)) AS days_old,
    CASE
        WHEN MONTH(dob) = MONTH(CAST(GETDATE() AS DATE)) THEN 'Birthday Month!'
        WHEN MONTH(dob) = MONTH(DATEADD(month, 1, CAST(GETDATE() AS DATE)))
            THEN 'Birthday Next Month'
        ELSE 'Birthday in ' + DATENAME(month, dob)
    END AS birthday_status
FROM students

```

Output:

```

+-----+-----+-----+-----+
| sname | dob       | days_old | birthday_status |
+-----+-----+-----+-----+
| Aarav | 2010-03-15 | 5114 | Birthday Month! |
| Vivaan | 2010-07-22 | 4985 | Birthday in July |
| Aditya | 2009-11-08 | 5241 | Birthday in November |
| Vihaan | 2010-01-30 | 5158 | Birthday in January |
| Arjun | 2010-09-12 | 4933 | Birthday in September |
| Sai | 2010-05-18 | 5049 | Birthday in May |
| Arnav | 2010-04-25 | 5073 | Birthday Next Month |
| Krishna | 2010-06-08 | 5029 | Birthday in June |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)

```

16. Best Practices

16.1 Date Function Best Practices

- Always store dates in standard formats (YYYY-MM-DD) for consistency

- Use DATEDIFF for day calculations, not manual arithmetic
- Account for leap years when calculating ages (divide by 365.25)
- Use FORMAT for display, not for comparisons
- Index date columns that are frequently used in WHERE clauses
- Use DATEADD with a negative number to subtract dates; there is no DATE_SUB in SQL Server

16.2 CASE Statement Best Practices

- Order conditions from most specific to least specific
- Always include an ELSE clause to handle unexpected values
- Use meaningful aliases for CASE expressions
- Consider extracting complex CASE logic into views for reusability
- Keep CASE statements readable by proper indentation

16.3 NULL Handling Best Practices

- Use IS NULL or IS NOT NULL, never = NULL or <> NULL
- Decide whether NULL or a default value is more appropriate for your use case
- Use COALESCE when you have multiple fallback options
- Use NULLIF to prevent errors like division by zero
- Document the meaning of NULL in your tables

16.4 Type Conversion Best Practices

- Validate data before conversion to avoid errors
- Use CAST for standard SQL compatibility
- Specify precision for DECIMAL conversions
- Be aware of data loss when converting to less precise types
- Test conversions with edge cases (empty strings, very large numbers, etc.)

17. Common Pitfalls

17.1 Date Arithmetic Errors

Adding or subtracting numbers directly to dates can produce unexpected results. Always use DATEADD:

```
-- Wrong
SELECT dob + 365 FROM students;

-- Correct
SELECT DATEADD(day, 365, dob) FROM students;
```

17.2 NULL Comparison Mistakes

Remember that `NULL = NULL` returns `NULL`, not `TRUE`:

```
-- Wrong
SELECT * FROM students WHERE marks = NULL;

-- Correct
SELECT * FROM students WHERE marks IS NULL;
```

17.3 CASE Statement Ordering

Conditions are evaluated in order. Placing a general condition before specific ones will prevent the specific ones from ever being reached:

```
-- Wrong (everything becomes 'Pass')
CASE
    WHEN marks >= 35 THEN 'Pass'
    WHEN marks >= 90 THEN 'Excellent'
END

-- Correct
CASE
    WHEN marks >= 90 THEN 'Excellent'
    WHEN marks >= 35 THEN 'Pass'
    ELSE 'Fail'
END
```

18. Module Summary

This module covered advanced SQL functions for date/time manipulation and conditional logic:

Date/Time Functions:

- `CAST(GETDATE() AS DATE)`, `CAST(GETDATE() AS TIME)`, `GETDATE()` - Get current date, time, and datetime

- **GETDATE()** vs **CURRENT_TIMESTAMP** - Both return current datetime; CURRENT_TIMESTAMP is ANSI standard and portable; GETDATE() is SQL Server only
- **GETUTCDATE(), SYSDATETIME(), SYSDATETIMEOFFSET()** - UTC and high-precision alternatives
- **UTC storage + front-end timezone conversion** - The recommended production pattern; store DATETIME2 in UTC, convert for display using AT TIME ZONE
- **YEAR, MONTH, DAY** - Extract date components
- **DATEPART(unit, date)** - Extract any date/time component using unit keywords (year, quarter, month, week, day, hour, minute, second, millisecond, etc.)
- **DATENAME(weekday/month, ...)** - Get names of days and months
- **DATEADD(unit, n, date)** - Add or subtract time intervals (use negative n to subtract)
- **DATEDIFF(unit, start, end)** - Calculate difference between dates in any unit
- **FORMAT(date, 'format')** - Format dates for display using .NET format strings

Conditional Logic:

- **CASE WHEN** - Implement if-else logic in queries
- **ISNULL** - Replace NULL with a default value (2 arguments)
- **COALESCE** - Return first non-NULL value (multiple arguments)
- **NULLIF** - Return NULL if two values are equal

Type Conversion:

- **CAST** - Convert data types (standard SQL syntax)
- **CONVERT** - Convert data types (alternative syntax)

These functions enable sophisticated data analysis, business logic implementation, and data formatting. When combined with string and numeric functions from the previous module, they provide comprehensive tools for data transformation and manipulation. In the next module, we will explore aggregate functions and grouping, which allow us to summarize and analyze data across multiple rows.

These functions enable sophisticated data analysis, business logic implementation, and data formatting. When combined with string and numeric functions from the previous module, they provide comprehensive tools for data transformation and manipulation. In the next module, we will explore aggregate functions and grouping, which allow us to summarize and analyze data across multiple rows.

19. Cross-Database Function Reference

The table below maps every function covered in this module to its equivalent across SQL Server (T-SQL), MySQL, Oracle, and PostgreSQL. Use this as a quick reference when porting queries or working across different database systems.

Legend: ☒ Native function exists |  Achieved differently | ☒ Not available natively

19.1 Current Date and Time

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
Current date only	<code>CAST(GETDATE() AS DATE)</code>	<code>CURDATE()</code> / <code>CURRENT_DATE</code>	<code>TRUNC(SYSDATE)</code>	<code>CURRENT_DATE</code>
Current time only	<code>CAST(GETDATE() AS TIME)</code>	<code>CURTIME()</code> / <code>CURRENT_TIME</code>	<code>TO_CHAR(SYSDAT E, 'HH24:MI:SS')</code>	<code>CURRENT_TIME</code>
Current date + time	<code>GETDATE()</code> / <code>CURRENT_TIMEST AMP</code>	<code>NOW()</code> / <code>CURRENT_TIMEST AMP</code>	<code>SYSDATE</code>	<code>NOW()</code> / <code>CURRENT_TIMEST AMP</code>
Current UTC date + time	<code>GETUTCDATE()</code> / <code>SYSUTCDATETIME ()</code>	<code>UTC_TIMESTAMP()</code>	<code>SYS_EXTRACT_UT C(SYSTIMESTAMP)</code>	<code>NOW()</code> AT TIME ZONE 'UTC'
High-precision datetime	<code>SYSDATETIME() (DATETIME2, 100ns)</code>	 <code>NOW(6)</code> (microseconds)	<code>SYSTIMESTAMP</code>	<code>CLOCK_TIMESTAM P()</code>
Datetime with timezone offset	<code>SYSDATETIMEOFF SET()</code>	☒ No native DATETIMEOFF SET type	<code>SYSTIMESTAMP (TIMESTAMP WITH TIME ZONE)</code>	<code>NOW()</code> with <code>TIMESTAMP TZ</code> type

19.2 Extracting Date/Time Components

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
Extract year	<code>YEAR(date)</code> / <code>DATEPART(year, date)</code>	<code>YEAR(date)</code>		<code>EXTRACT(YEAR FROM date)</code> / <code>DATE_PART('yea r', date)</code>

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
			EXTRACT(YEAR FROM date) / TO_NUMBER(TO_C HAR(date, 'YYYY ''))	
Extract month	MONTH(date) / DATEPART(month , date)	MONTH(date)	EXTRACT(MONTH FROM date)	EXTRACT(MONTH FROM date) / DATE_PART('mon th', date)
Extract day	DAY(date) / DATEPART(day, date)	DAY(date)	EXTRACT(DAY FROM date)	EXTRACT(DAY FROM date) / DATE_PART('day ', date)
Extract hour	DATEPART(hour, datetime)	HOUR(datetime)	EXTRACT(HOUR FROM datetime)	EXTRACT(HOUR FROM datetime) / DATE_PART('hou r', datetime)
Extract minute	DATEPART(minut e, datetime)	MINUTE(datetim e)	EXTRACT(MINUTE FROM datetime)	EXTRACT(MINUTE FROM datetime) / DATE_PART('min ute', datetime)
Extract second	DATEPART(secon d, datetime)	SECOND(datetim e)	EXTRACT(SECOND FROM datetime)	EXTRACT(SECOND FROM datetime) / DATE_PART('sec ond', datetime)
Extract quarter	DATEPART(quant er, date)	QUARTER(date)	EXTRACT(QUARTE R FROM date) (Oracle 21c+) / 	EXTRACT(QUARTE R FROM date) / DATE_PART('qua rter', date)

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
			<code>CEIL(EXTRACT(MONTH FROM date)/3)</code>	
Extract week number	<code>DATEPART(week, date) / DATEPART(iso_week, date)</code>	<code>WEEK(date) / WEEKOFYEAR(date)</code>	<code>TO_CHAR(date, 'IW') (ISO week)</code>	<code>EXTRACT(WEEK FROM date) / DATE_PART('week', date)</code>
Extract day of year	<code>DATEPART(dayofyear, date)</code>	<code>DAYOFYEAR(date)</code>	<code>TO_NUMBER(TO_CHAR(date, 'DDD'))</code>	<code>EXTRACT(DOY FROM date) / DATE_PART('doy', date)</code>
Day of week (number)	<code>DATEPART(weekday, date)</code>	<code>DAYOFWEEK(date) (1=Sun) / WEEKDAY(date) (0=Mon)</code>	<code>TO_CHAR(date, 'D') (1=Sun)</code>	<code>EXTRACT(DOW FROM date) (0=Sun) / EXTRACT(ISODOW FROM date) (1=Mon)</code>

19.3 Day and Month Names

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
Day name (e.g. "Friday")	<code>DATENAME(weekday, date)</code>	<code>DAYNAME(date)</code>	<code>TO_CHAR(date, 'DAY')</code>	<code>TO_CHAR(date, 'Day')</code>
Month name (e.g. "March")	<code>DATENAME(month, date)</code>	<code>MONTHNAME(date)</code>	<code>TO_CHAR(date, 'MONTH')</code>	<code>TO_CHAR(date, 'Month')</code>

19.4 Adding and Subtracting Date Intervals

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
Add N days	<code>DATEADD(day, N, date)</code>	<code>DATE_ADD(date, INTERVAL N DAY) / date + INTERVAL N DAY</code>	<code>date + N</code> (plain arithmetic)	<code>date + INTERVAL 'N days' / date + N</code>
Add N months				

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
	<code>DATEADD(month, N, date)</code>	<code>DATE_ADD(date, INTERVAL N MONTH)</code>	<code>ADD_MONTHS(date, N)</code>	<code>date + INTERVAL 'N months'</code>
Add N years	<code>DATEADD(year, N, date)</code>	<code>DATE_ADD(date, INTERVAL N YEAR)</code>	<code>ADD_MONTHS(date, N*12)</code>	<code>date + INTERVAL 'N years'</code>
Add N hours	<code>DATEADD(hour, N, datetime)</code>	<code>DATE_ADD(datetime, INTERVAL N HOUR)</code>	<code>datetime + N/24</code>	<code>datetime + INTERVAL 'N hours'</code>
Add N minutes	<code>DATEADD(minute, N, datetime)</code>	<code>DATE_ADD(datetime, INTERVAL N MINUTE)</code>	<code>datetime + N/1440</code>	<code>datetime + INTERVAL 'N minutes'</code>
Subtract N days	<code>DATEADD(day, -N, date)</code>	<code>DATE_SUB(date, INTERVAL N DAY) / date - INTERVAL N DAY</code>	<code>date - N</code>	<code>date - INTERVAL 'N days' / date - N</code>

19.5 Date Difference

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
Difference in days	<code>DATEDIFF(day, start, end)</code>	<code>DATEDIFF(end, start)</code> ⚠ args reversed, days only	<code>end - start</code> (plain subtraction returns days)	<code>DATE_PART('day', end - start) / (end::date - start::date)</code>
Difference in months	<code>DATEDIFF(month, start, end)</code>	📅 <code>PERIOD_DIFF(DATE_FORMAT(end, '%Y%m'), DATE_FORMAT(start, '%Y%m'))</code>	<code>MONTHS_BETWEEN(end, start)</code>	<code>DATE_PART('year', age(end, start)) * 12 + DATE_PART('month', age(end, start))</code>
Difference in years	<code>DATEDIFF(year, start, end)</code>	📅 <code>YEAR(end) - YEAR(start)</code>		<code>DATE_PART('year', age(end, start))</code>

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
			 <code>FLOOR(MONTHS_BETWEEN(end, start) / 12)</code>	
Difference in hours	<code>DATEDIFF(hour, start, end)</code>	 <code>TIMESTAMPDIFF(HOUR, start, end)</code>	 <code>(end - start) * 24</code>	<code>DATE_PART('epoch', end - start) / 3600</code>
Difference in minutes	<code>DATEDIFF(minute, start, end)</code>	 <code>TIMESTAMPDIFF(MINUTE, start, end)</code>	 <code>(end - start) * 1440</code>	<code>DATE_PART('epoch', end - start) / 60</code>
Age as human-readable interval	 <code>FLOOR(DATEDIFF(day, dob, GETDATE()) / 365.25)</code>	 <code>TIMESTAMPDIFF(YEAR, dob, NOW())</code>	 <code>FLOOR(MONTHS_BETWEEN(SYSDATE, dob) / 12)</code>	 <code>AGE(end, start) →</code> returns interval like "14 years 2 mons"

19.6 Date Formatting

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
Format date as string	<code>FORMAT(date, 'MMMM dd, yyyy')</code>	<code>DATE_FORMAT(date, '%M %d, %Y')</code>	<code>TO_CHAR(date, 'MONTH DD, YYYY')</code>	<code>TO_CHAR(date, 'Month DD, YYYY')</code>
Format as DD/MM/YYYY	<code>FORMAT(date, 'dd/MM/yyyy')</code>	<code>DATE_FORMAT(date, '%d/%m/%Y')</code>	<code>TO_CHAR(date, 'DD/MM/YYYY')</code>	<code>TO_CHAR(date, 'DD/MM/YYYY')</code>
Format with time	<code>FORMAT(datetime, 'dd/MM/yyyy HH:mm')</code>	<code>DATE_FORMAT(datetime, '%d/%m/%Y %H:%i')</code>	<code>TO_CHAR(datetime, 'DD/MM/YYYY HH24:MI')</code>	<code>TO_CHAR(datetime, 'DD/MM/YYYY HH24:MI')</code>
Parse string to date	<code>CAST('2024-03-15' AS DATE) / CONVERT(DATE, '2024-03-15')</code>	<code>STR_TO_DATE('15/03/2024', '%d/%m/%Y')</code>	<code>TO_DATE('15/03/2024', 'DD/MM/YYYY')</code>	<code>TO_DATE('15/03/2024', 'DD/MM/YYYY')</code>

19.7 Timezone Handling

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
Current UTC time	<code>GETUTCDATE()</code>	<code>UTC_TIMESTAMP()</code>	<code>SYS_EXTRACT_UT C(SYSTIMESTAMP)</code>	<code>NOW() AT TIME ZONE 'UTC'</code>
Convert between timezones	<code>datetime AT TIME ZONE 'UTC' AT TIME ZONE 'India Standard Time'</code>	<code>CONVERT_TZ(dat etime, 'UTC', 'Asia/ Kolkata')</code>	<code>FROM_TZ(CAST(d atetime AS TIMESTAMP), 'UTC') AT TIME ZONE 'Asia/ Kolkata'</code>	<code>datetime AT TIME ZONE 'UTC' AT TIME ZONE 'Asia/ Kolkata'</code>
Timezone-aware data type	<code>DATETIMEOFFSE T</code>	✗ None (workaround: store UTC + separate tz column)	<code>TIMESTAMP WITH TIME ZONE</code>	<code>TIMESTAMPTZ (TIMESTAMP WITH TIME ZONE)</code>
Extract UTC offset	<code>DATEPART(tzoff set, SYSDATETIMEOFF SET()) → minutes</code>	📦 No direct equivalent	<code>EXTRACT(TIMEZO NE_HOUR FROM SYSTIMESTAMP)</code>	<code>EXTRACT(TIMEZO NE FROM now()) → seconds</code>

19.8 NULL Handling

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
Replace NULL with default (2 args)	<code>ISNULL(expr, default)</code>	<code>IFNULL(expr, default)</code>	<code>NVL(expr, default)</code>	✗ No ISNULL; use <code>COALESCE(expr, default)</code>
First non-NULL from a list	<code>COALESCE(a, b, c, ...)</code>	<code>COALESCE(a, b, c, ...)</code>	<code>COALESCE(a, b, c, ...) / NVL2(expr, val_if_not_nul l, val_if_null)</code>	<code>COALESCE(a, b, c, ...)</code>
	<code>NULLIF(a, b)</code>	<code>NULLIF(a, b)</code>	<code>NULLIF(a, b)</code>	<code>NULLIF(a, b)</code>

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
Return NULL if two values are equal				
If-else NULL check (3 args)	 Use <code>CASE WHEN expr IS NOT NULL THEN ... ELSE ... END</code>	<code>IF(expr IS NOT NULL, val_true, val_false) / IFNULL</code>	<code>NVL2(expr, val_if_not_null, val_if_null)</code>	 Use <code>CASE WHEN ... END</code>

19.9 Conditional Logic

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
If-else logic	<code>CASE WHEN ... THEN ... ELSE ... END</code>	<code>CASE WHEN ... END / IF(cond, true_val, false_val)</code>	<code>CASE WHEN ... END / DECODE(expr, val1, result1, ..., default)</code>	<code>CASE WHEN ... END</code>
Simple two-way condition	 <code>IIF(condition, true_val, false_val)</code> (SQL Server 2012+)	<code>IF(condition, true_val, false_val)</code>	<code>DECODE</code> or <code>CASE</code>	 Use <code>CASE WHEN ... END</code>

19.10 Type Conversion

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
Standard type cast	<code>CAST(expr AS INT)</code>	<code>CAST(expr AS UNSIGNED) / CAST(expr AS SIGNED)</code>	<code>CAST(expr AS NUMBER)</code>	<code>CAST(expr AS INTEGER) / expr::INTEGER</code>
Convert function	<code>CONVERT(INT, expr)</code> — type first	<code>CONVERT(expr, UNSIGNED)</code> — value first ⚠️ reversed		 Use <code>CAST</code> or <code>::</code> shorthand

Purpose	SQL Server	MySQL	Oracle	PostgreSQL
			<code>TO_NUMBER(expr)</code> <code>) /</code> <code>TO_CHAR(expr) /</code> <code>TO_DATE(expr,</code> <code>fmt)</code>	
To integer	<code>CAST(expr AS INT) /</code> <code>CAST(expr AS BIGINT)</code>	<code>CAST(expr AS UNSIGNED) /</code> <code>CAST(expr AS SIGNED)</code>	<code>CAST(expr AS INTEGER) /</code> <code>TO_NUMBER(expr)</code>	<code>CAST(expr AS INTEGER) /</code> <code>expr::INT</code>
To decimal/float	<code>CAST(expr AS DECIMAL(10,2))</code>	<code>CAST(expr AS DECIMAL(10,2))</code>	<code>CAST(expr AS NUMBER(10,2))</code>	<code>CAST(expr AS NUMERIC(10,2)) /</code> <code>expr::NUMERIC(10,2)</code>
To string/varchar	<code>CAST(expr AS VARCHAR(50)) /</code> <code>CONVERT(VARCHAR, expr)</code>	<code>CAST(expr AS CHAR) /</code> <code>CONVERT(expr, CHAR)</code>	<code>TO_CHAR(expr) /</code> <code>CAST(expr AS VARCHAR2(50))</code>	<code>CAST(expr AS TEXT) /</code> <code>expr::TEXT</code>
To date	<code>CAST(expr AS DATE) /</code> <code>CONVERT(DATE, expr)</code>	<code>CAST(expr AS DATE) /</code> <code>STR_TO_DATE(expr, fmt)</code>	<code>TO_DATE(expr, 'YYYY-MM-DD')</code>	<code>CAST(expr AS DATE) /</code> <code>expr::DATE</code>

19.11 Key Syntax Differences to Watch Out For

Gotcha	SQL Server	MySQL	Oracle	PostgreSQL
DATEDIFF argument order	<code>DATEDIFF(unit, start, end)</code>	<code>DATEDIFF(end, start)</code> ⚠ reversed	N/A — use subtraction	N/A — use subtraction or <code>AGE()</code>
CONVERT argument order	<code>CONVERT(type, expr)</code>	<code>CONVERT(expr, type)</code> ⚠ reversed	Not used (use <code>TO_CHAR</code> / <code>TO_NUMBER</code> / <code>TO_DATE</code>)	Not typically used (prefer <code>CAST</code> or <code>::</code>)
Integer type name		<code>UNSIGNED</code> / <code>SIGNED</code> / <code>INT</code>	<code>NUMBER</code> / <code>INTEGER</code>	

Gotcha	SQL Server	MySQL	Oracle	PostgreSQL
	<code>INT</code> / <code>BIGINT</code> (no UNSIGNED)			<code>INTEGER</code> / <code>BIGINT</code> (no UNSIGNED)
String concatenation	<code>str1 + str2</code>	<code>CONCAT(str1, str2)</code> (+ does addition)	<code>str1 str2</code> / <code>CONCAT()</code>	<code>str1 str2</code> / <code>CONCAT()</code>
Limit rows returned	<code>SELECT TOP N ...</code>	<code>... LIMIT N</code>	<code>FETCH FIRST N ROWS ONLY</code> (12c+) / <code>ROWNUM <= N</code>	<code>... LIMIT N</code>
CURRENT_TIM ESTAMP syntax	<code>CURRENT_TIMEST AMP</code> (no parentheses)	<code>CURRENT_TIMEST AMP</code> or <code>CURRENT_TIMEST AMP()</code>	<code>CURRENT_TIMEST AMP</code> (no parentheses)	<code>CURRENT_TIMEST AMP</code> (no parentheses)